

LAPORAN PRATIKUM STRUKTUR DATA
“QUEUE”

disusun Oleh:
AFIF RAHMAN SALEH
NIM 251153105

Dosen Pengampu: Dr. WAHYUDI, S.T, M.T
Asisten Praktikum : M Galid Alvero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN
2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas rahmat-Nya sehingga laporan praktikum Struktur Data ini dapat diselesaikan. Laporan ini disusun sebagai bentuk pertanggungjawaban kegiatan praktikum pekan pertama dengan topik implementasi Stack pada java.

Ucapan terima kasih penulis sampaikan kepada dosen pengampu, asisten praktikum, serta rekan-rekan mahasiswa atas bimbingan dan kerja sama yang diberikan. Penulis menyadari laporan ini masih jauh dari sempurna, sehingga kritik dan saran sangat diharapkan demi perbaikan di masa mendatang.

Semoga laporan ini bermanfaat bagi penulis maupun pembaca dalam memahami konsep struktur data dan penerapannya dalam pemrograman.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Landasan Teori	2
2.2 Kode Java yang Dipraktikkan	2
2.3 Penjelasan Langkah Kerja.....	8
2.4 Analisis Hasil	12
BAB III KESIMPULAN.....	15
5.1 Kesimpulan Praktikum.....	15
DAFTAR PUSTAKA	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data tidak selalu dapat dilakukan secara statis menggunakan array. Array memiliki keterbatasan, seperti ukuran yang tetap dan sulitnya melakukan operasi penyisipan atau penghapusan di tengah struktur. Oleh karena itu, dibutuhkan struktur data dinamis yang lebih fleksibel.

Salah satu solusi adalah Linked List, khususnya Doubly Linked List (DLL). DLL memungkinkan setiap node terhubung dua arah, yaitu ke node berikutnya (*next*) dan node sebelumnya (*prev*). Dengan kemampuan ini, DLL lebih efisien dalam operasi penyisipan, penghapusan, dan penelusuran dibandingkan Single Linked List.

Praktikum ini dilakukan untuk memberikan pengalaman langsung kepada mahasiswa dalam mengimplementasikan DLL menggunakan bahasa pemrograman Java, sehingga konsep pointer dua arah dan manipulasi node dapat dipahami secara praktis.

1.2 Tujuan Praktikum

1. Memahami Memahami konsep dasar Doubly Linked List.
2. Mempelajari cara membuat node dengan pointer next dan prev.
3. Mengimplementasikan operasi dasar DLL: insert, delete, traversal..

1.3 Manfaat Praktikum

1. Memberikan pemahaman mendalam tentang pengelolaan data dinamis dengan pointer dua arah.
2. Membantu mahasiswa menyelesaikan permasalahan pemrograman yang membutuhkan akses maju dan mundur.
3. Melatih keterampilan praktis dalam menulis kode yang efisien.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Linked List adalah struktur data dinamis yang terdiri dari simpul-simpul (node) yang saling terhubung. Linked list terdapat single dan doubly. Doubly Linked List (DLL) adalah jenis linked list di mana setiap node memiliki dua pointer yaitu next untuk menunjuk ke node berikutnya dan Prev untuk menunjuk ke node sebelumnya.

DLL memiliki beberapa keunggulan dibanding Single Linked List (SLL) yaitu DLL dapat menelusuri dari dua arah (maju dan mundur) dan operasi penghapusan dan penyisipan lebih fleksibel.

Berikut beberapa Operasi dasar dalam DLL:

1. Insert: menambahkan node di awal, akhir, atau posisi tertentu.
2. Delete: menghapus node di awal, akhir, atau posisi tertentu.
3. Traversal: menelusuri node dari head ke tail (forward) atau tail ke head (backward).

Contoh penereapan DLL dalam aplikasi nyata seperti Undo/Redo pada editor teks, navigation history pada browser, dan playlist musik yang bisa maju/mundur antar lagu.

2.2 Kode Java yang Dipraktikkan

1. Kelas NodeDLL_2511531005

```
• package pekan6_2511531005;
•
• public class NodeDLL_2511531005 {
•
•     int data_1005;
•     NodeDLL_2511531005 next_1005;
•     NodeDLL_2511531005 prev_1005;
• }
```

```

•         public NodeDLL_2511531005(int data_1005) {
•             this.data_1005 = data_1005;
•             this.next_1005 = null;
•             this.prev_1005 = null;
•         }
•     }
• }

```

2. Kelas InsertDLL_2511531005

```

• package pekan6_2511531005;
•
• public class InsertDLL_2511531005 {
•
•     static NodeDLL_2511531005
InsertBegin_1005(NodeDLL_2511531005 head_1005, int data_1005) {
•         NodeDLL_2511531005 new_node_1005 = new
NodeDLL_2511531005(data_1005);
•
•         new_node_1005.next_1005 = head_1005;
•
•         if(head_1005 != null) {
•             head_1005.prev_1005 = new_node_1005;
•         }
•         return new_node_1005;
•     }
•
•     public static NodeDLL_2511531005
InsertEnd_1005(NodeDLL_2511531005 head_1005, int newData) {
•
•         NodeDLL_2511531005 newNode_1005 = new
NodeDLL_2511531005(newData);
•
•         if (head_1005 == null) {
•             head_1005 = newNode_1005;
•         }
•         else {
•             NodeDLL_2511531005 curr = head_1005;
•             while (curr.next_1005 != null) {
•                 curr = curr.next_1005;
•             }
•             curr.next_1005 = newNode_1005;
•             newNode_1005.prev_1005 = curr;
•         }
•         return head_1005;
•     }
•
•     public static NodeDLL_2511531005
InsertAtPosition_1005(NodeDLL_2511531005 head_1005, int pos_1005,
int new_data_1005) {
•

```

```

        NodeDLL_2511531005 new_node_1005 = new
NodeDLL_2511531005(new_data_1005);

        if(pos_1005 == 1 ) {
            new_node_1005.next_1005 = head_1005;
            if(head_1005 != null) {
                head_1005.prev_1005 = new_node_1005;
            }
            head_1005 = new_node_1005;
            return head_1005;
        }

        NodeDLL_2511531005 curr_1005 = head_1005;

        for (int i = 1; i < pos_1005-1 && curr_1005 != null;
++i) {
            curr_1005 = curr_1005.next_1005;
        }
        if(curr_1005 == null) {
            System.out.println("posisi tidak ada");
            return head_1005;
        }
        new_node_1005.prev_1005 = curr_1005;
        new_node_1005.next_1005 = curr_1005.next_1005;
        curr_1005.next_1005 = new_node_1005;

        if(new_node_1005.next_1005 != null) {
            new_node_1005.next_1005.prev_1005 =
new_node_1005;
        }
        return head_1005;
    }

    public static void printList_1005(NodeDLL_2511531005
head_1005) {

        NodeDLL_2511531005 curr_1005 = head_1005;

        while (curr_1005 != null) {
            System.out.print(curr_1005.data_1005 + " <->
");
            curr_1005 = curr_1005.next_1005;
        }
        System.out.println();
    }

    public static void main(String[] args) {

        NodeDLL_2511531005 head_1005 = new
NodeDLL_2511531005(2);
        head_1005.next_1005 = new NodeDLL_2511531005(3);
        head_1005.next_1005.prev_1005 = head_1005;
    }

```

```

    head_1005.next_1005.next_1005 = new
NodeDLL_2511531005(5);
    head_1005.next_1005.next_1005.prev_1005 =
head_1005.next_1005;

    System.out.println("DLL awal : ");
    printList_1005(head_1005);

    head_1005 = InsertBegin_1005(head_1005, 1);
    System.out.print("simpul 1 ditambahkan di awal : ");
    printList_1005(head_1005);

    System.out.print("simpul 6 ditambahkan di akhir :
");
    int data_1005 = 6;
    head_1005 = InsertEnd_1005(head_1005, data_1005);
    printList_1005(head_1005);

    System.out.print("tambah node 4 di posisi 4 : ");
    int data2_1005 = 4;
    int pos_1005 = 4;
    head_1005 = InsertAtPosition_1005(head_1005,
pos_1005, data2_1005);
    printList_1005(head_1005);
}
}

```

3. Kelas PenelusuranDLL_2511531005

```

package pekan6_2511531005;

public class PenelusuranDLL_2511531005 {

    static void forwardTraversal_1005(NodeDLL_2511531005
head_1005) {
        NodeDLL_2511531005 curr_1005 = head_1005;

        while (curr_1005 != null) {
            System.out.print(curr_1005.data_1005 + " <->
");
            curr_1005 = curr_1005.next_1005;
        }
        System.out.println();
    }

    static void backwardTraversal_1005(NodeDLL_2511531005
tail_1005) {
        NodeDLL_2511531005 curr_1005 = tail_1005;

        while (curr_1005 != null) {

```

```

    System.out.print(curr_1005.data_1005 + " <->
");
    curr_1005 = curr_1005.prev_1005;
}
System.out.println();

}

public static void main(String[] args) {

    NodeDLL_2511531005 head_1005 = new
NodeDLL_2511531005(1);
    NodeDLL_2511531005 second_1005 = new
NodeDLL_2511531005(2);
    NodeDLL_2511531005 third_1005 = new
NodeDLL_2511531005(3);

    head_1005.next_1005 = second_1005;
    second_1005.prev_1005 = head_1005;
    second_1005.next_1005 = third_1005;
    third_1005.prev_1005 = second_1005;

    System.out.println("penelusuran maju : ");
    forwardTraversal_1005(head_1005);

    System.out.println("penelusuran mundur : ");
    backwardTraversal_1005(third_1005);

}

}

```

4. Kelas HapusDLL_2511531005

```

package pekan6_2511531005;

public class HapusDLL_2511531005 {

    public static NodeDLL_2511531005 delHead_1005(NodeDLL_2511531005
head_1005) {
        if(head_1005 == null) {
            return null;
        }
        NodeDLL_2511531005 temp_1005 = head_1005;
        head_1005 = head_1005.next_1005;
        if(head_1005 != null) {
            head_1005.prev_1005 = null;
        }
        return head_1005;
    }

    public static NodeDLL_2511531005 delLast_1005(NodeDLL_2511531005
head_1005) {
        if(head_1005 == null) {

```

```

        return null;
    }
    if (head_1005.next_1005 == null) {
        return null;
    }
    NodeDLL_2511531005 curr_1005 = head_1005;
    while (curr_1005.next_1005 != null) {
        curr_1005 = curr_1005.next_1005;
    }

    if(curr_1005.prev_1005 != null) {
        curr_1005.prev_1005.next_1005 = null;
    }
    return head_1005;
}

public static NodeDLL_2511531005 delPos_1005(NodeDLL_2511531005
head_1005, int pos_1005) {

    if(head_1005 == null) {
        return head_1005;
    }
    NodeDLL_2511531005 curr_1005 = head_1005;

    for (int i = 1; curr_1005 != null && i < pos_1005; ++i) {
        curr_1005 = curr_1005.next_1005;
    }

    if(curr_1005 == null) {
        return head_1005;
    }

    if(curr_1005.prev_1005 != null) {
        curr_1005.prev_1005.next_1005 = curr_1005.next_1005;
    }

    if(curr_1005.next_1005 != null) {
        curr_1005.next_1005.prev_1005 = curr_1005.prev_1005;
    }

    if(head_1005 == curr_1005) {
        head_1005 = curr_1005.next_1005;
    }
    return head_1005;
}

public static void printList_1005(NodeDLL_2511531005 head_1005) {
    NodeDLL_2511531005 curr_1005 = head_1005;
    while (curr_1005 != null) {
        System.out.print(curr_1005.data_1005 + " <-> ");
        curr_1005 = curr_1005.next_1005;
    }
    System.out.println();
}

public static void main(String[] args) {

```

```

        NodeDLL_2511531005 head_1005 = new NodeDLL_2511531005(1);
        head_1005.next_1005 = new NodeDLL_2511531005(2);
        head_1005.next_1005.prev_1005 = head_1005;
        head_1005.next_1005.next_1005 = new NodeDLL_2511531005(3);
        head_1005.next_1005.next_1005.prev_1005 =
head_1005.next_1005;
        head_1005.next_1005.next_1005.next_1005 = new
NodeDLL_2511531005(4);
        head_1005.next_1005.next_1005.next_1005.prev_1005=
head_1005.next_1005.next_1005;
        head_1005.next_1005.next_1005.next_1005.next_1005 = new
NodeDLL_2511531005(5);

        head_1005.next_1005.next_1005.next_1005.next_1005.prev_1005=
head_1005.next_1005.next_1005.next_1005;

        System.out.print("DLL awal : ");
        printList_1005(head_1005);

        System.out.print("setelah head dihapus : ");
        head_1005 = delHead_1005(head_1005);
        printList_1005(head_1005);

        System.out.print("seteleah node terakhir dihapus : ");
        head_1005 = delLast_1005(head_1005);
        printList_1005(head_1005);

        System.out.print("menghapus node ke 2 : ");
        head_1005 = delPos_1005(head_1005, 2);

        printList_1005(head_1005);

    }
}

```

2.3 Penjelasan Langkah Kerja

1. Kelas NodeDLL_2511531005

- class NodeDLL_2511531005 → untuk mendefinisikan struktur node DLL.
- int data_1005 → untuk menyimpan nilai data (tipe integer).
- NodeDLL_2511531005 next_1005 → pointer ke node berikutnya.
- NodeDLL_2511531005 prev_1005 → pointer ke node sebelumnya.
- Konstruktor

```
2. public NodeDLL_2511531005(int data_1005) {
```

```

3.         this.data_1005 = data_1005;
4.         this.next_1005 = null;
5.         this.prev_1005 = null;
6.     }

```

setiap kali node baru dibuat, data diisi sesuai parameter, sedangkan next dan prev diinisialisasikan dengan null.

2. Kelas InsertDLL_2511531005

- Method utama
 - InsertBegin_1005

```

3. static NodeDLL_2511531005 InsertBegin_1005(NodeDLL_2511531005
   head_1005, int data_1005) {
4.     NodeDLL_2511531005 new_node_1005 = new
   NodeDLL_2511531005(data_1005);
5.
6.     new_node_1005.next_1005 = head_1005;
7.
8.     if(head_1005 != null) {
9.         head_1005.prev_1005 = new_node_1005;
10.    }
11.    return new_node_1005;
12. }

```

- Untuk membuat node baru di awal
- new_node.next = head → node baru menunjuk ke head lama.
- Jika head lama tidak null, maka head.prev = new_node.
- Node baru menjadi head baru.

- InsertEnd_1005

```

13. public static NodeDLL_2511531005
   InsertEnd_1005(NodeDLL_2511531005 head_1005, int newData) {
14.
15.     NodeDLL_2511531005 newNode_1005 = new
   NodeDLL_2511531005(newData);
16.
17.     if (head_1005 == null) {
18.         head_1005 = newNode_1005;
19.     }
20.     else {
21.         NodeDLL_2511531005 curr = head_1005;
22.         while (curr.next_1005 != null) {
23.             curr = curr.next_1005;
24.         }

```

```

25.         curr.next_1005 = newNode_1005;
26.         newNode_1005.prev_1005 = curr;
27.     }
28.     return head_1005;
29. }

```

- Untuk membuat node baru di akhir
- Jika list kosong (head == null), node baru jadi head.
- Jika tidak, iterasi sampai node terakhir (curr.next == null).
- Sambungkan node baru di akhir (curr.next = newNode), dan set newNode.prev = curr.
- InsertAtPosition_1005
 - Untuk membuat node baru di posisi tertentu
 - Jika posisi = 1, sama seperti insert di awal.
 - Jika posisi > 1, iterasi sampai node ke (pos-1).
 - Jika posisi valid, node baru disisipkan di antara curr dan curr.next.
 - Update pointer prev dan next agar konsisten.
- printList_1005

```

public static void printList_1005(NodeDLL_2511531005 head_1005) {
    NodeDLL_2511531005 curr_1005 = head_1005;

    while (curr_1005 != null) {
        System.out.print(curr_1005.data_1005 + " <-> ");
        curr_1005 = curr_1005.next_1005;
    }
    System.out.println();
}

```

- untuk menampilkan isi DLL dari head ke tail.
- Main method
 - Membuat DLL awal: 2 <-> 3 <-> 5.
 - Insert node 1 di awal.
 - Insert node 6 di akhir.
 - Insert node 4 di posisi ke-4.

3. Kelas PenelusuranDLL_2511531005

- Method utama
 - forwardTraversal_1005

```
static void forwardTraversal_1005(NodeDLL_2511531005 head_1005) {  
    NodeDLL_2511531005 curr_1005 = head_1005;  
  
    while (curr_1005 != null) {  
        System.out.print(curr_1005.data_1005 + " <-> ");  
        curr_1005 = curr_1005.next_1005;  
    }  
    System.out.println();  
}
```

- dengan memulai dari head.
- Lalu Iterasi ke next hingga null.
- Cetak data setiap node.

- backwardTraversal_1005

```
static void backwardTraversal_1005(NodeDLL_2511531005 tail_1005) {  
  
    NodeDLL_2511531005 curr_1005 = tail_1005;  
  
    while (curr_1005 != null) {  
        System.out.print(curr_1005.data_1005 + " <-> ");  
        curr_1005 = curr_1005.prev_1005;  
    }  
    System.out.println();  
}
```

- dengan memulai dari tail.
- Lalu Iterasi ke prev hingga null.
- Cetak data setiap node.

- Main method
 - Membuat DLL sederhana: 1 <-> 2 <-> 3.
 - Traversal maju dari head.
 - Traversal mundur dari tail

4. Kelas HapusDLL_2511531005

- Method utama
 - delHead_1005

- untuk menghapus node di awal(head)
 - Jika list kosong, return null.
 - Head digeser ke node berikutnya (head = head.next).
 - Jika head baru tidak null, set head.prev = null.
 - delLast_1005
 - untuk menghapus node diakhir(tail)
 - Jika list kosong, return null.
 - Jika hanya satu node, return null (list jadi kosong).
 - Iterasi sampai node terakhir.
 - Putuskan hubungan node terakhir dengan prev.
 - delPos_1005
 - untuk menghapus node di posisi tertentu
 - Iterasi sampai posisi yang diminta.
 - Jika node ditemukan:
 - Hubungkan prev dengan next.
 - Hubungkan next dengan prev.
 - Jika node yang dihapus adalah head, geser head ke node berikutnya.
 - printList_1005
 - untuk menampilkan isi DLL.
- Main method
 - Membuat DLL awal: 1 <-> 2 <-> 3 <-> 4 <-> 5.
 - Hapus head.
 - Hapus node terakhir.
 - Hapus node ke-2.

2.4 Analisis Hasil

Berdasarkan program pada semua kelas yang dipraktikkan pada pekan ini, diperoleh beberapa hasil sebagai berikut:

1. Kelas NodeDLL_2511531005

Berdasarkan kode program di atas, node ini adalah komponen dasar DLL untuk mendefinisikan struktur. Semua operasi insert, delete, dan traversal akan menggunakan node ini sebagai unit data.

2. Kelas InsertDLL_2511531005

Output :

```
<terminated> InsertDLL_2511531005 java Application C:\Users\OSER\Downloads\ eclipse
DLL awal :
2 <-> 3 <-> 5 <->
simpul 1 ditambahkan di awal : 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambahkan di akhir : 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4 : 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Berdasarkan output di atas, kode program di atas berhasil karena DLL dapat ditambahkan di awal, di akhir, dan di posisi tertentu

3. Kelas PenelusuranDLL_2511531005

Output :

```
<terminated> PenelusuranDLL_251
penelusuran maju :
1 <-> 2 <-> 3 <->
penelusuran mundur :
3 <-> 2 <-> 1 <->
```

Berdasarkan output di atas, kode program di atas berhasil menampilkan penelusuran maju yang menunjukkan dari arah head ke tail. Sedangkan penelusuran mundur menunjukkan arah tail ke head. Hal ini menunjukkan DLL terbukti dua arah.

4. Kelas HapusDLL_2511531005

Output :

```
DLL awal : 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
setelah head dihapus : 2 <-> 3 <-> 4 <-> 5 <->
seteleah node terakhir dihapus : 2 <-> 3 <-> 4 <->
menghapus node ke 2 : 2 <-> 4 <->
```

Berdasarkan output di atas, kode program tersebut berhasil menampilkan kondisi ketika head di hapus, node terakhir di hapus dan node tertentu ketika dihapus.

BAB III

KESIMPULAN

1.1 Kesimpulan Praktikum

Berdasarkan hasil praktikum di atas dapat disimpulkan bahwa DLL adalah struktur data yang fleksibel karena setiap node memiliki pointer next dan prev, sehingga memungkinkan penelusuran dua arah. Melalui implementasi kode, operasi dasar seperti penyisipan, penghapusan, dan penelusuran dapat dilakukan dengan baik tanpa merusak keterhubungan antar node. Hal ini menunjukkan pentingnya pengelolaan pointer agar list tetap konsisten setelah setiap operasi. Secara keseluruhan, mahasiswa memahami bahwa DLL merupakan pondasi penting dalam pengelolaan data dinamis dan dapat diaplikasikan pada berbagai kasus nyata seperti sistem antrian, navigasi riwayat, maupun fitur undo/redo.

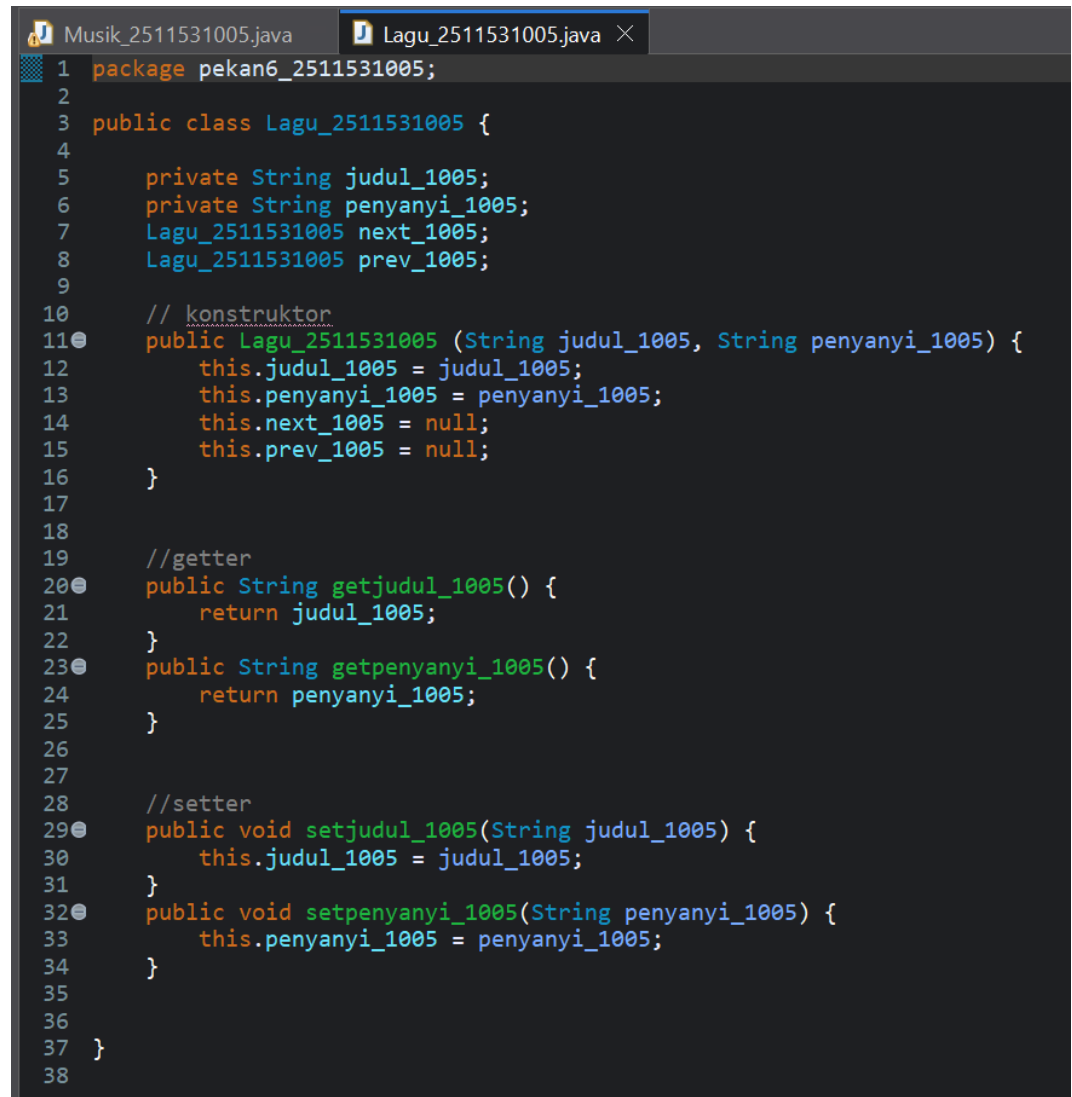
DAFTAR PUSTAKA

- [1] Oracle. (2025). *Java Platform, Standard Edition Documentation*. Oracle Corporation.
- [2] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.

TUGAS PRAKTIKUM PEKAN 4

A. Kode program

1. Lagu_2511531005



```
1 package pekan6_2511531005;
2
3 public class Lagu_2511531005 {
4
5     private String judul_1005;
6     private String penyanyi_1005;
7     Lagu_2511531005 next_1005;
8     Lagu_2511531005 prev_1005;
9
10    // konstruktor
11    public Lagu_2511531005 (String judul_1005, String penyanyi_1005) {
12        this.judul_1005 = judul_1005;
13        this.penyanyi_1005 = penyanyi_1005;
14        this.next_1005 = null;
15        this.prev_1005 = null;
16    }
17
18
19    //getter
20    public String getjudul_1005() {
21        return judul_1005;
22    }
23    public String getpenyanyi_1005() {
24        return penyanyi_1005;
25    }
26
27
28    //setter
29    public void setjudul_1005(String judul_1005) {
30        this.judul_1005 = judul_1005;
31    }
32    public void setpenyanyi_1005(String penyanyi_1005) {
33        this.penyanyi_1005 = penyanyi_1005;
34    }
35
36
37 }
38
```

2. Musik_2511531005

```
Musik_2511531005.java Lagu_2511531005.java
1 package pekan6_2511531005;
2 import java.util.Scanner;
3
4
5 public class Musik_2511531005 {
6
7     private Lagu_2511531005 head_1005;
8     private Lagu_2511531005 tail_1005;
9
10    //tambah lagu di akhir playlist
11    public void tambahLagu_1005(String judul_1005, String penyanyi_1005) {
12
13        //membuat node baru
14        Lagu_2511531005 newNode_1005 = new Lagu_2511531005 (judul_1005, penyanyi_1005);
15
16        //jika playlist kosong, head dan tail menunjuk ke node baru
17        if(head_1005 == null) {
18            head_1005 = newNode_1005;
19            tail_1005 = newNode_1005;
20        }
21        else {
22            tail_1005.next_1005 = newNode_1005;
23            newNode_1005.prev_1005 = tail_1005;
24            tail_1005 = newNode_1005;
25        }
26        System.out.println("lagu berhasil ditambahkan ");
27    }
28
29    //tambah menghapus lagu di awal playlist
30    public void hapusLaguAwal_1005() {
31        if(head_1005 == null) {
32            System.out.println("playlist kosong");
33            return;
34        }
35        System.out.println("menghapus lagu " + head_1005.getjudul_1005());
36
37        if(head_1005 == tail_1005) {
38            head_1005 = null;
39            tail_1005 = null;
40        }
41        else {
42            head_1005 = head_1005.next_1005;
43
44            else {
45                head_1005 = head_1005.next_1005;
46                head_1005.prev_1005 = null;
47            }
48            System.out.println("lagu berhasil dihapus");
49        }
50
51        //lihat playlist maju
52        public void lihatPlaylistMaju_1005() {
53            if(head_1005 == null) {
54                System.out.println("playlist kosong");
55                return;
56            }
57            System.out.println("=== playlist maju ===");
58
59            Lagu_2511531005 curr_1005 = head_1005 ;
60
61            while (curr_1005 != null) {
62                System.out.println(curr_1005.getjudul_1005()+ " - " + curr_1005.getpenyanyi_1005());
63                curr_1005 = curr_1005.next_1005;
64            }
65
66            //lihat playlist mundur
67            public void lihatPlaylistMundur() {
68                if(tail_1005 == null) {
69                    System.out.println("playlist kosong");
70                    return;
71                }
72                System.out.println("=== playlist mundur ===");
73                Lagu_2511531005 curr_1005 = tail_1005;
74                while (curr_1005 != null) {
75                    System.out.println(curr_1005.getjudul_1005() + " - " + curr_1005.getpenyanyi_1005());
76                    curr_1005 = curr_1005.prev_1005;
77                }
78            }
79
80            //cari lagu
81            public void cariLagu_2511531005 (String judul_1005) {
82                if(head_1005 == null) {
83                    System.out.println("playlist kosong");
```

```

80 //cari lagu
81 public void cariLagu_2511531005 (String judul_1005) {
82     if(head_1005 == null) {
83         System.out.println("playlist kosong");
84         return;
85     }
86     Lagu_2511531005 curr_1005 = head_1005;
87     boolean ketemu = false;
88
89     while (curr_1005 != null) {
90         if(curr_1005.getJudul_1005().equalsIgnoreCase(judul_1005)) {
91             System.out.println("lagu ditemukan " + curr_1005.getJudul_1005() + " - " + curr_1005.getPenyanyi_1005());
92             ketemu = true;
93             break;
94         }
95         curr_1005 = curr_1005.next_1005;
96     }
97     if (!ketemu) {
98         System.out.println("lagu tidak ditemukan!");
99     }
100 }
101
102 public static void main(String[] args) {
103
104     Scanner input = new Scanner (System.in);
105     Musik_2511531005 playlist = new Musik_2511531005();
106     int choice;
107
108     do {
109         System.out.println("\n== playlist musik nim : 2511531005");
110         System.out.println("1. tambah lagu");
111         System.out.println("2. hapus lagu pertama");
112         System.out.println("3. lihat playlist maju");
113         System.out.println("4. lihat playlist mundur ");
114         System.out.println("5. cari lagu");
115         System.out.println("6. keluar");
116         System.out.println("pilihan : ");
117         choice = input.nextInt();
118         input.nextLine();
119
120     } switch (choice) {
121

```

```

79
80 //cari lagu
81 public void cariLagu_2511531005 (String judul_1005) {
82     if(head_1005 == null) {
83         System.out.println("playlist kosong");
84         return;
85     }
86     Lagu_2511531005 curr_1005 = head_1005;
87     boolean ketemu = false;
88
89     while (curr_1005 != null) {
90         if(curr_1005.getJudul_1005().equalsIgnoreCase(judul_1005)) {
91             System.out.println("lagu ditemukan " + curr_1005.getJudul_1005()
92             + " - " + curr_1005.getPenyanyi_1005());
93             ketemu = true;
94             break;
95         }
96         curr_1005 = curr_1005.next_1005;
97     }
98     if (!ketemu) {
99         System.out.println("lagu tidak ditemukan!");
100     }
101 }
102
103 public static void main(String[] args) {
104
105     Scanner input = new Scanner (System.in);
106     Musik_2511531005 playlist = new Musik_2511531005();
107     int choice;
108
109     do {
110         System.out.println("\n== playlist musik nim : 2511531005");
111         System.out.println("1. tambah lagu");
112         System.out.println("2. hapus lagu pertama");
113         System.out.println("3. lihat playlist maju");
114         System.out.println("4. lihat playlist mundur ");
115         System.out.println("5. cari lagu");
116         System.out.println("6. keluar");
117         System.out.println("pilihan : ");
118         choice = input.nextInt();
119         input.nextLine();
120

```

B. output

1. Tambah Lagu

```
=== playlist musik nim : 2511531005
1. tambah lagu
2. hapus lagu pertama
3. lihat playlist meaju
4. lihat playlist mundur
5. cari lagu
6. keluar
pilihan :
1
judul :
perang telah usai
penyanyi :
nadin amizah
lagu berhasil ditambahkan
```

```
0. keluar
pilihan :
1
judul :
oh tuan
penyanyi :
hindia
lagu berhasil ditambahkan
```

```
1
judul :
sorai
penyanyi :
nadin amizah
lagu berhasil ditambahkan
```

```
pilihan :
1
judul :
tarot
penyanyi :
feast
lagu berhasil ditambahkan
```

2. Hapus Lagu Pertama

```
=== playlist musik nim : 2511531005
1. tambah lagu
2. hapus lagu pertama
3. lihat playlist meaju
4. lihat playlist mundur
5. cari lagu
6. keluar
pilihan :
2
menghapus lagu perang telah usai
lagu berhasil dihapus
```

3. Lihat Playlist (Maju)

```

=== playlist musik nim : 2511531005
1. tambah lagu
2. hapus lagu pertama
3. lihat playlist meaju
4. lihat playlist mundur
5. cari lagu
6. keluar
pilihan :
3
=== playlist maju ===
oh tuan - hindia
sorai - nadin amizah
tarot - feast

```

4. Lihat Playlist (Mundur)

```

=== playlist musik nim : 2511531005
1. tambah lagu
2. hapus lagu pertama
3. lihat playlist meaju
4. lihat playlist mundur
5. cari lagu
6. keluar
pilihan :
4
=== playlist mundur ===
tarot - feast
sorai - nadin amizah
oh tuan - hindia

```

5. Cari Lagu

```

=== playlist musik nim : 2511531005
1. tambah lagu
2. hapus lagu pertama
3. lihat playlist meaju
4. lihat playlist mundur
5. cari lagu
6. keluar
pilihan :
5
masukkan judul :
tarot
lagu ditemukan tarot - feast

```

```

pilihan :
5
masukkan judul :
perang telah usai
lagu tidak ditemukan!

```

6. Keluar

```
=== playlist musik nim : 2511531005
1. tambah lagu
2. hapus lagu pertama
3. lihat playlist meaju
4. lihat playlist mundur
5. cari lagu
6. keluar
pilihan :
6
program selesai
```